
Tecnologie del Linguaggio Naturale

Domande e Risposte – Parte 1 (Mazzei)

Università degli Studi di Torino (UniTO)
Corso di Laurea Magistrale in Informatica
A.A. 2025/26

Alessandro Olivero

Documento generato a scopo di studio

0. Indice

1	Come funziona il Transition Based Parser (MALT)?	2
1.1	Configurazione	2
1.2	Le tre operazioni (Arc-Standard)	2
1.3	Esempio: “I booked a morning flight”	2
1.4	Oracle e classificatore	2
2	Spiegare l’uso delle HMM nel PoS Tagging	3
2.1	Le 3 fasi (terminologia delle slide)	3
2.2	Modelling: dalla probabilità congiunta a HMM	3
2.3	Esempio guida: disambiguare “race” (slide Jurafsky/Mazzei)	3
2.4	Decoding: perché non si enumerano tutte le sequenze	4
2.5	Limiti dell’HMM ed estensioni	4
2.6	Performance	4
3	Che differenza c’è tra MEMM e HMM?	4
3.1	Generativo vs. Discriminativo (definizione delle slide)	5
3.2	MEMM – Maximum Entropy Markov Model	5
3.3	Le 2 ipotesi del MEMM	5
3.4	Esempi di feature usate (a differenza dell’HMM)	5
3.5	Label Bias Problem	5
4	Cosa è una Probabilistic CFG (PCFG)?	6
4.1	Probabilità di un albero sintattico	6
4.2	Utilizzo: disambiguation sintattico	6
4.3	Limitazioni	6
4.4	Lexicalized PCFG (Collins 1997, Charniak 1997)	7
5	Cosa è il task dell’Aggregation (NLG)?	7
5.1	Contesto: i 7 task della pipeline NLG (dominio di esempio: i treni)	7
5.2	Esempio applicativo: BabyTalk	8
6	I 5 algoritmi per il parsing delle dipendenze sintattiche	8
6.1	Nota sulla proiettività	9
7	Il formalismo Lambda-FoL per la Semantica Computazionale	9
7.1	Principio di composizionalità (Frege)	9
7.2	Lambda astrazione e beta-riduzione	9
7.3	Grammatica semantica di base	9
7.4	Determinatori e quantificatori	9
7.5	Integrazione con la sintassi	10
7.6	Semantica di Montague	10
7.7	NLTK	10
8	La rappresentazione Neo-Davidsoniana	10
8.1	Semantica Davidsoniana (base)	10
8.2	Approccio Neo-Davidsoniano	11
8.3	Vantaggi	11

9	Le due ipotesi fondamentali della linguistica computazionale	11
9.1	Esempio guida (slide Mazzei): quando finisce una frase?	12
9.2	Sintesi	12
10	La Frame-Based Semantics nei Dialogue Systems (DS)	12
10.1	Struttura di un frame	12
10.2	Algoritmo in 3 passi (pipeline NLU)	13
10.3	Gestione del dialogo (Dialogue Manager)	13
10.4	Condition-Action Rules sugli slot	13
10.5	Sistemi multi-frame e Frame Detection	13
10.6	Limiti dei frame	14
11	Fare un esempio di ambiguità puramente semantica	14
11.1	Polisemia lessicale	14
11.2	Ambiguità di scope quantificazionale	14
11.3	Ambiguità del pronome (coreference)	14
11.4	Esempio classico (Mazzei)	14
12	Ambiguità sintattica: PP attachment e coordinazione	14
12.1	PP Attachment (Attacco del Sintagma Preposizionale)	14
12.2	Ambiguità della Coordinazione	15
12.3	Crescita esponenziale delle analisi (ambiguità strutturale)	15
13	Come funziona l'algoritmo CKY?	16
13.1	CKY nell'anatomia del parser (Steedman)	16
13.2	Perché bottom-up e non top-down? (Razionalisti vs. Empiricisti, slide Mazzei)	16
13.3	Programmazione Dinamica (Bellman, 1957)	16
13.4	Pre-requisito: CNF	17
13.5	Struttura dati: tabella triangolare	17
13.6	Esempio: "Book that flight" (CKY)	17
14	Complessità dell'algoritmo CKY	17
14.1	Analisi dei loop	17
14.2	Versione probabilistica (PCKY)	18
14.3	Confronto con altri algoritmi (slide Mazzei)	18
15	Dipendenze vs. Costituenti	18
15.1	Vantaggi delle dipendenze (Tesnière 1959)	19
15.2	Vantaggi dei costituenti	19
15.3	Convertibilità	19
16	La misura PARSEVAL	19
16.1	Definizione	19
16.2	Labeled vs. Unlabeled	19
16.3	Crossing Brackets	20
16.4	Precision e Recall per le PCFG (in pratica)	20
16.5	Limitazioni	20
17	Cosa è un Treebank?	20

17.1	Caratteristiche	20
17.2	Esempi principali	21
17.3	Universal Dependencies (UD): dettagli (slide Mazzei)	21
17.4	Utilizzo	21
18	HMM: modello generativo o discriminativo?	21
18.1	Spiegazione	21
18.2	Confronto con i modelli discriminativi	22
19	Spiegare cosa si intende per Anatomia di un Parser	22
19.1	Relazione tra i tre componenti	23
19.2	Esempi	23
20	I 6 fenomeni notevoli del dialogo tra umani	23
21	Cosa è il PoS Tagging?	24
21.1	A cosa serve (usi pratici, dalle slide)	25
21.2	Classi principali e tagset	25
21.3	Ambiguità e sfide	25
21.4	Approccio Rule-Based: ENGTWOL	25
21.5	Approcci statistici e neurali	26
21.6	Valutazione	26
22	Cosa è il NER Tagging?	26
22.1	Classi più comuni (slide Mazzei/Jurafsky)	26
22.2	Perché NER è più difficile del PoS Tagging	26
22.3	Schema di annotazione BIO	27
22.4	Approcci	27
22.5	Perché serve il NER	27

1. Come funziona il Transition Based Parser (MALT)?

Il **MALT Parser** (Maltparser, Nivre et al. 2008) è un parser per la sintassi a **dipendenze** che modella il parsing come una sequenza di *transizioni* su una configurazione.

Configurazione

La configurazione è una tripla (σ, β, A) :

- **Stack** σ : contiene i token già parzialmente processati (inizia con il solo ROOT).
- **Buffer** β : contiene i token ancora da processare (inizia con l'intera frase).
- **Insieme di archi** A : le relazioni di dipendenza costruite finora (inizia vuoto).

Stato iniziale: $\{[\text{root}], [w_1, \dots, w_n], \emptyset\}$

Stato finale: $\{[\text{root}], [], A_{\text{finale}}\}$

Le tre operazioni (Arc-Standard)

- **SHIFT**: sposta il primo elemento del buffer in cima allo stack.
- **LEFT-ARC**(r): crea un arco $\text{top}(\sigma) \rightarrow \text{sec}(\sigma)$ con relazione r ; rimuove $\text{sec}(\sigma)$ dallo stack. (Il top diventa testa del secondo.)
- **RIGHT-ARC**(r): crea un arco $\text{sec}(\sigma) \rightarrow \text{top}(\sigma)$ con relazione r ; rimuove il top dallo stack. (Il secondo diventa testa del top.)

Esempio: “I booked a morning flight”

Stack	Buffer	Azione	Archi
[root]	[I, booked, a, morning, flight]	SHIFT	–
[root, I]	[booked, a, morning, flight]	SHIFT	–
[root, I, booked]	[a, morning, flight]	LEFT-ARC(Subj)	booked←a
[root, booked]	[a, morning, flight]	SHIFT	–
[root, booked, a]	[morning, flight]	SHIFT	–
[root, booked, a, morning]	[flight]	SHIFT	–
[root, booked, a, morning, flight]	[]	LEFT-ARC(Mod)	flight←morning
[root, booked, a, flight]	[]	LEFT-ARC(Det)	flight←a
[root, booked, flight]	[]	RIGHT-ARC(Obj)	booked→flight
[root, booked]	[]	RIGHT-ARC(Root)	root→booked

Oracle e classificatore

In fase di **training** un *oracle* (basato su un gold treebank) indica la transizione corretta. A runtime un **classificatore** (SVM, perceptron, rete neurale) predice la

prossima azione sulla base di feature estratte dalla configurazione corrente (PoS in cima allo stack, forme, relazioni già costruite, ecc.).

La complessità è **lineare** $O(n)$: ogni token viene shiftato e rimosso esattamente una volta.

2. Spiegare l'uso delle HMM nel PoS Tagging

Il PoS Tagging si inquadra come un problema di **sequence labelling**: data una frase (sequenza di osservazioni) $w_1, \dots, w_n$, si vuole trovare la sequenza di tag $t_1, \dots, t_n$ più probabile, cioè $\hat{t}_{1:n} = \arg \max_{t_{1:n}} P(t_{1:n} \mid w_{1:n})$.

Le 3 fasi (terminologia delle slide)

- **Modelling**: dare un modello formale del problema.
- **Learning**: un algoritmo per stimare i parametri del modello da un corpus annotato.
- **Decoding**: un algoritmo per applicare il modello e calcolare il risultato (la sequenza di tag).

Modelling: dalla probabilità congiunta a HMM

Tramite la **regola di Bayes**, $P(t_{1:n} \mid w_{1:n})$ si trasforma in un prodotto di probabilità più facili da stimare:

$$\hat{t}_{1:n} = \arg \max_{t_{1:n}} \underbrace{P(t_1) \prod_{i=2}^n P(t_i \mid t_{i-1})}_{\text{prior}} \cdot \underbrace{\prod_{i=1}^n P(w_i \mid t_i)}_{\text{likelihood}}$$

Un HMM è quindi definito da: **stati nascosti** (i tag), **osservazioni** (le parole), **probabilità di transizione** $P(t_i \mid t_{i-1})$ (Tag→Tag, es. un determinante è seguito spesso da aggettivo/nome) e **probabilità di emissione** $P(w_i \mid t_i)$ (Tag→parola, es. VBZ “emette” spesso “is”). Si stimano entrambe per conteggio (MLE) su un corpus annotato a mano (*Learning*).

Esempio guida: disambiguare “race” (slide Jurafsky/Mazzei)

“*Secretariat is expected to **race** tomorrow*” vs. “*the **race** for outer space*”: nel primo caso “race” è verbo (VB), nel secondo è nome (NN). Confrontando le due sequenze di tag candidate dopo “to” (TO):

$$\begin{array}{rcl} P(\text{VB}|\text{TO}) \cdot P(\text{NR}|\text{VB}) \cdot P(\text{race}|\text{VB}) & = & 0.83 \times 0.0027 \times 0.00012 = 2.7 \times 10^{-7} \\ P(\text{NN}|\text{TO}) \cdot P(\text{NR}|\text{NN}) \cdot P(\text{race}|\text{NN}) & = & 0.00047 \times 0.0012 \times 0.00057 = 3.2 \times 10^{-10} \end{array}$$

Il prodotto con VB è ~1000 volte più alto: il modello sceglie **correttamente** il tag verbo per “race” in questo contesto, nonostante $P(\text{race}|\text{NN})$ da solo sia più alto di $P(\text{race}|\text{VB})$ – è il *contesto* (la transizione da TO) a fare la differenza.

Decoding: perché non si enumerano tutte le sequenze

Con ~ 30 tag nel Penn Treebank tagset e una frase media di 20 parole, enumerare tutte le sequenze possibili richiederebbe 30^{20} valutazioni: intrattabile. Si usa quindi l'algoritmo di **Viterbi** (programmazione dinamica): l'idea chiave è che la probabilità della miglior sequenza di tag che termina nello stato j al tempo t si scompone nella probabilità della miglior sequenza fino a $t - 1$, moltiplicata per la probabilità di transizione verso j e la probabilità di emissione dell'osservazione corrente – non serve ricordare tutti i cammini, solo il migliore per ogni cella. Complessità: $O(n \cdot |T|^2)$.

Esercizio tipico (slide Mazzei)

Dato il mini-corpus annotato: *Paolo/N pesca/V, Giovanni/N ama/V i/D cani/N, Francesca/N ama/N, Una/D pesca/N Francesca/A* – si chiede di fare il *learning* dei parametri dell'HMM (contando transizioni ed emissioni) e poi il *decoding* a mano con Viterbi della frase “*Paolo ama Francesca*”. È un buon esercizio per verificare di aver capito l'algoritmo passo-passo.

Limiti dell'HMM ed estensioni

L'HMM classico (bigram, $P(t_i|t_{i-1})$) soffre di **sparsità dei dati** se si estende a trigram ($P(t_i|t_{i-2}, t_{i-1})$): si combinano i due modelli con un'**interpolazione lineare pesata** (*deleted interpolation*). Il problema strutturale dell'HMM è che non sfrutta **feature ricche**: per parole sconosciute (non viste in training) può solo affidarsi a suffissi e capitalizzazione codificati indirettamente nelle probabilità di emissione, perché ogni nuova fonte di conoscenza deve essere forzata dentro $P(w|t)$ o $P(t|t-1)$ – da qui la necessità di MEMM e CRF.

Performance

La baseline più semplice (*most frequent tag + unknown = noun*) raggiunge già il 92% di accuratezza. I modelli moderni (HMM, CRF, BERT fine-tuned) raggiungono circa il 97% su inglese – prestazioni simili tra loro, in parte perché molte parole sono comunque non ambigue (l'articolo “the”, la punteggiatura, ecc. fanno già guadagnare molti punti).

3. Che differenza c'è tra MEMM e HMM?

Aspetto	HMM	MEMM
Tipo di modello	Generativo	Discriminativo
Distribuzione	$P(w, t)$ congiunta	$P(t w)$ condizionale
Feature	Solo parola corrente	Feature arbitrarie sul contesto
Training	Conteggi + smoothing	Ottimizzazione (L-BFGS, SGD)
Label bias	No	Sì (problema noto)
Normalizzazione	Globale	Locale per ogni stato

Generativo vs. Discriminativo (definizione delle slide)

Citando direttamente Jurafsky (slide Mazzei): un **modello generativo** è addestrato a generare i dati x a partire dalla classe y – il termine di likelihood $P(x|y)$ esprime che, data la classe y , si predice quali feature aspettarsi nell’input x (è il modello dell’HMM). Un **modello discriminativo** calcola direttamente $P(y|x)$, discriminando tra i possibili valori della classe y senza passare prima per una likelihood.

MEMM – Maximum Entropy Markov Model

Il **Maximum Entropy modeling** (Maxent) combina un insieme eterogeneo di feature in un modello probabilistico: le feature impongono dei vincoli al modello, e si cerca la distribuzione più uniforme possibile (massima entropia) compatibile con quei vincoli. Il MEMM modella direttamente la distribuzione condizionale con un modello *log-lineare*:

$$P(t_i \mid t_{i-1}, w_{1:n}) = \frac{1}{Z} \exp \left(\sum_k \lambda_k f_k(t_i, t_{i-1}, w_{1:n}, i) \right)$$

Il training si fa per **conditional maximum likelihood estimation**, equivalente a una **regressione logistica multinomiale**: si scelgono i pesi λ_k che massimizzano la log-probabilità condizionale dei dati di training.

Le 2 ipotesi del MEMM

Il MEMM fa due assunzioni semplificative (slide Mazzei):

1. t_i è condizionalmente indipendente da tutte le osservazioni e etichette precedenti, dato w_i e t_{i-1} .
2. Le osservazioni w_i sono indipendenti tra loro.

Esempi di feature usate (a differenza dell’HMM)

A differenza dell’HMM, che usa solo $P(w_i|t_i)$ e $P(t_i|t_{i-1})$, il MEMM/Maxent può usare feature arbitrarie sulla singola parola (anche senza memoria del contesto): prefissi (*unfathomable*: un- $\rightarrow$ JJ), suffissi (*Importantly*: -ly $\rightarrow$ RB), capitalizzazione (*Meridian*: CAP $\rightarrow$ NNP), word-shape (*35-year*: d-x $\rightarrow$ JJ), presenza di numeri/trattini/maiuscole, ecc. Con sole feature “a parola singola” (senza memoria) Maxent raggiunge 93.7% di accuratezza complessiva e 82.6% sulle parole sconosciute – già un buon risultato, che mostra il vantaggio delle feature morfologiche per le parole non viste in training.

Label Bias Problem

Il principale svantaggio del MEMM è il **label bias problem**: le probabilità di transizione vengono normalizzate *localmente* per ogni stato (la somma delle probabilità uscenti da uno stato è 1, indipendentemente da quanti successori ha), favorendo gli stati con pochi successori indipendentemente dall’evidenza globale osservata. I **Conditional Random Fields** (CRF) risolvono questo problema con una normalizzazione *globale* sulla sequenza intera, invece che stato per stato.

4. Cosa è una Probabilistic CFG (PCFG)?

Una **Probabilistic Context-Free Grammar** (PCFG) estende una CFG associando una probabilità a ogni regola di produzione.

Una PCFG è una quintupla $G = (N, \Sigma, R, S, P)$ dove:

- N : insieme dei simboli non-terminali.
- Σ : insieme dei simboli terminali (vocabolario).
- R : insieme delle regole di produzione $A \rightarrow \alpha$.
- $S \in N$: simbolo iniziale.
- P : funzione $P(A \rightarrow \alpha) \in [0, 1]$ con $\sum_{\alpha} P(A \rightarrow \alpha) = 1$ per ogni A .

Probabilità di un albero sintattico

La probabilità di un albero T è il *prodotto* delle probabilità di tutte le regole utilizzate:

$$P(T) = \prod_{(A \rightarrow \alpha) \in T} P(A \rightarrow \alpha)$$

Utilizzo: disambiguation sintattico

Tra tutti gli alberi $T_1, T_2, \dots$ compatibili con la grammatica per una frase data, si sceglie quello con probabilità massima: $\hat{T} = \arg \max_T P(T)$. I parametri si stimano da un treebank con MLE:

$$P(A \rightarrow \alpha) = \frac{C(A \rightarrow \alpha)}{C(A)}$$

Treebank Grammar (slide Mazzei): dato un piccolo treebank con due alberi – uno per “Paolo ama Francesca” ($S \rightarrow NPVP$, $VP \rightarrow VN$) e uno per “Paolo corre” ($S \rightarrow NPVP$, $VP \rightarrow V$) – si stimano via conteggio: $P(S \rightarrow NPVP) = 2/2 = 1$, $P(NP \rightarrow N) = 2/2 = 1$, $P(VP \rightarrow VN) = 1/2 = .5$, $P(VP \rightarrow V) = 1/2 = .5$, $P(N \rightarrow \text{Paolo}) = 2/3 = .66$, $P(N \rightarrow \text{Francesca}) = 1/3 = .33$, $P(V \rightarrow \text{corre}) = 1/2 = .5$, $P(V \rightarrow \text{ama}) = 1/2 = .5$. Questo è esattamente il meccanismo MLE applicato a un treebank reale (seppur minuscolo).

Limitazioni

Le PCFG assumono l'**ipotesi di indipendenza**: la probabilità di una regola non dipende dal contesto (dal non-terminale padre, dalla posizione nella frase, dalle parole effettive coinvolte), quindi **non ci sono preferenze lessicali**. Questo produce stime poco accurate quando la scelta strutturale dipende fortemente dal significato delle parole (tipicamente nel PP attachment).

Lexicalized PCFG (Collins 1997, Charniak 1997)

Le **Lexicalized PCFG** mitigano il problema potenziando ogni regola CF con l'informazione sulla *head lessicale* di ciascun costituente coinvolto:

$$A \rightarrow BC \quad \Longrightarrow \quad A(\text{head}_A) \rightarrow B(\text{head}_B) C(\text{head}_C)$$

Questo è anche il **punto di unione tra formalismo a costituenti e formalismo a dipendenze**: la head lessicale di un costituente corrisponde alla testa nella rappresentazione a dipendenze. Esempio numerico dalle slide – per la regola $VP \rightarrow VBD\ NP\ PP$ lessicalizzata su *dumped*, le probabilità del PP variano a seconda del nome scelto per NP e della preposizione:

Regola lessicalizzata	Probabilità
$VP(\text{dumped}) \rightarrow VBD(\text{dumped})\ NP(\text{sacks})\ PP(\text{into})$	3×10^{-10}
$VP(\text{dumped}) \rightarrow VBD(\text{dumped})\ NP(\text{cats})\ PP(\text{into})$	8×10^{-11}
$VP(\text{dumped}) \rightarrow VBD(\text{dumped})\ NP(\text{hats})\ PP(\text{into})$	4×10^{-10}
$VP(\text{dumped}) \rightarrow VBD(\text{dumped})\ NP(\text{sacks})\ PP(\text{above})$	1×10^{-12}

Si nota che “dumped ... sacks ... into” ha probabilità molto più alta di “dumped ... sacks ... above”: il modello ha imparato dal corpus che si *dump* qualcosa *into* un contenitore più spesso che *above* – una vera preferenza lessicale, impossibile da catturare con una PCFG non lessicalizzata.

5. Cosa è il task dell'Aggregation (NLG)?

L'**Aggregation** (o *Sentence Aggregation*) è il terzo dei sette task della pipeline NLG simbolica di Mazzei. Risponde al problema: se esprimessimo ogni messaggio separatamente, uno per frase, il discorso risulterebbe poco naturale e non fluente. L'aggregation **combina più messaggi in frasi più complesse**.

È il **primo task che dipende dalla lingua specifica** (i task precedenti, Content Determination e Discourse Planning, sono indipendenti dalla lingua di output).

Esempio (dominio dei treni, slide Mazzei):

Messaggi separati: *IDENTITY(NextTrain, CaledonianExpress)* e *DEPARTURETIME(CaledonianExpress, 10:00)*.

Senza aggregation: “The next train is the Caledonian Express. The Caledonian Express leaves at 10am.”

Con aggregation: “**The next train, which leaves at 10am, is the Caledonian Express.**”

Rischio: l'aggregazione può creare ambiguità; il sistema assume che il lettore sappia disambiguare correttamente.

Contesto: i 7 task della pipeline NLG (dominio di esempio: i treni)

1. Content Determination
2. Discourse Planning

3. **Sentence Aggregation** ← questo task
4. Lexicalization
5. Referring Expression Generation
6. Syntactic/Morphological Realization
7. Orthographic Realization

Esempio applicativo: BabyTalk

BabyTalk è un sistema NLG reale a scopo medico, citato da Mazzei come caso di studio: genera automaticamente resoconti testuali a partire da dati di sensori e osservazioni dell'équipe medica in terapia intensiva neonatale, indirizzati a destinatari diversi con output differenziati: **BT-45** (45 minuti di dati per i dottori), **BT-Nurse** (12 ore per le infermiere), **BT-Family** (24 ore in linguaggio semplice per le famiglie). Lo stesso dato sorgente viene trasformato in testi diversi a seconda del pubblico.

6. I 5 algoritmi per il parsing delle dipendenze sintattiche

Secondo Mazzei (lezione 6), i cinque approcci principali al dependency parsing sono:

1. Dynamic Programming

Adattamento degli algoritmi a programmazione dinamica per le dipendenze. Nel caso generale la complessità è $O(n^5)$; con l'algoritmo di **Eisner (1996)**, che sfrutta la vincolo di proiettività, scende a $O(n^3)$. Garantisce l'ottimo globale per alberi proiettivi.

2. Graph Algorithms (MST – Maximum Spanning Tree)

Il parsing è formulato come ricerca del *Maximum Spanning Tree* (o *Minimum Spanning Tree* con pesi invertiti) su un grafo completo pesato dei token. Il punteggio di ogni arco $h \rightarrow d$ è assegnato da un modello ML (*arc-factored*). L'algoritmo di **Chu-Liu/Edmonds** trova l'albero ottimale anche per strutture non proiettive. Usato in **MSTParser** (McDonald et al. 2005) – il primo parser a dipendenze di Google.

3. Constituency + Conversion

Si esegue prima il parsing a costituenti (CFG/PCFG) con algoritmi standard (CKY, Earley), poi si converte l'albero a costituenti in un albero a dipendenze tramite *head-percolation tables* (tabelle che specificano quale figlio è la testa per ogni categoria). Approccio classico, meno usato nella pratica moderna.

4. Constraint Satisfaction

Le dipendenze vengono generate da un insieme di ipotesi candidate e poi filtrate/eliminate tramite *vincoli hard* (morfosintattici, di subcategorizzazione, semantici). Usato nel parser **FIDO/TUP** (Lesmo & Torasso 1985, Torino) e in **Constraint Grammar** (Karlsson 1990).

5. Transition-Based Parsing (MALT)

Approccio greedy basato su una sequenza di transizioni (LEFT-ARC, RIGHT-

ARC, SHIFT) guidata da un classificatore ML. Complessità **lineare** $O(n)$. Vedi Sezione 1 per la descrizione dettagliata.

Nota sulla proiettività

Un albero a dipendenze è **proiettivo** se ogni nodo “copre” uno span contiguo della frase (nessun arco si incrocia). Il MALT Arc-Standard produce solo alberi proiettivi; per il non-proiettivo si usa Arc-Eager o algoritmi di post-processing. MST gestisce nativamente il caso non-proiettivo.

7. Il formalismo Lambda-FoL per la Semantica Computazionale

Il **Lambda Calcolo applicato alla Logica del Primo Ordine** (*Lambda-FoL*) è il formalismo standard per costruire rappresentazioni semantiche composizionali a partire dalla struttura sintattica.

Principio di composizionalità (Frege)

Il significato di un'espressione è funzione del significato delle sue parti e del modo in cui sono combinate. Il lambda calcolo fornisce il meccanismo di combinazione.

Lambda astrazione e beta-riduzione

- **Lambda astrazione:** $\lambda x. \varphi(x)$ è una funzione che, dato x , restituisce $\varphi(x)$.
- **Beta-riduzione:** $(\lambda x. \varphi(x))(a) \Rightarrow \varphi[x \mapsto a]$

Grammatica semantica di base

Lessico:

NP: paolo $\rightsquigarrow \lambda P. P(\text{paolo})$

NP: francesca $\rightsquigarrow \lambda P. P(\text{francesca})$

V: ama $\rightsquigarrow \lambda y. \lambda x. \text{love}(x, y)$

Regole composizionali:

VP = V(NP) S = NP(VP)

Esempio – “Paolo ama Francesca”:

$$\text{VP} = (\lambda y. \lambda x. \text{love}(x, y))(\lambda P. P(\text{francesca}))$$

$$\Rightarrow \lambda x. \text{love}(x, \text{francesca})$$

$$\text{S} = (\lambda P. P(\text{paolo}))(\lambda x. \text{love}(x, \text{francesca}))$$

$$\Rightarrow \text{love}(\text{paolo}, \text{francesca})$$

Determinatori e quantificatori

Per gestire “un uomo ama Francesca” si usa il *type-raising* dei determinatori:

DT: $\text{un} \rightsquigarrow \lambda P. \lambda Q. \exists z (P(z) \wedge Q(z))$
 DT: $\text{ogni} \rightsquigarrow \lambda P. \lambda Q. \forall z (P(z) \Rightarrow Q(z))$
 DT: $\text{nessun} \rightsquigarrow \lambda P. \lambda Q. \forall z (P(z) \Rightarrow \neg Q(z))$

Esempio – “Un uomo ama Francesca”:

$$(\lambda P. \lambda Q. \exists z (P(z) \wedge Q(z))) (\lambda z. \text{man}(z)) \Rightarrow \lambda Q. \exists z (\text{man}(z) \wedge Q(z))$$

Applicato al VP $\lambda x. \text{love}(x, \text{francesca})$:

$$\exists z (\text{man}(z) \wedge \text{love}(z, \text{francesca}))$$

Integrazione con la sintassi

Il formalismo si integra con grammatiche categoriali (CCG) o con regole composizionali bottom-up su alberi sintattici. A ciascun lessema si assegna un'entrata *semantica* in Lambda-FoL; il significato dell'intera frase si ottiene componendo dal basso verso l'alto.

Semantica di Montague

Richard Montague propose un collegamento diretto tra linguaggi naturali e logica, rifiutando l'idea che ci sia una differenza qualitativa importante tra linguaggi formali e linguaggi naturali (entrambi possono essere trattati con gli stessi strumenti logico-formali). Montague introdusse anche una componente di **logica temporale**, per modellare l'influenza del tempo sul significato, scomponendo l'enunciato in tre componenti che si combinano: *U* (*utterance*, il momento dell'enunciazione), *R* (*reference*, il momento di riferimento) ed *E* (*event*, il momento dell'evento).

NLTK

NLTK (Natural Language ToolKit) è una libreria Python per la linguistica computazionale, utilizzata anche a scopo didattico, che supporta tra l'altro logica del primo ordine e lambda calcolo, theorem proving, costruzione di modelli e model checking. Esempio di semantica computazionale con NLTK (slide Mazzei): dato il parser caricato da una grammatica a feature (**simple-sem.fcfig**) e la frase “*Angus gives a bone to every dog*”, il sistema produce la rappresentazione semantica:

$$\forall z_2 (\text{dog}(z_2) \rightarrow \exists z_1 (\text{bone}(z_1) \wedge \text{give}(\text{angus}, z_1, z_2)))$$

8. La rappresentazione Neo-Davidsoniana

La rappresentazione **Neo-Davidsoniana** (Parsons 1990) estende la semantica Davidsoniana classica per gestire in modo uniforme argomenti verbali obbligatori e modificatori facoltativi.

Semantica Davidsoniana (base)

Davidson (1967) propose di introdurre una **variabile evento** *e* come argomento implicito del verbo, permettendo di trattare gli avverbi come predicati sull'evento. Esempio dalle slide Mazzei – “Paolo ama Francesca dolcemente”:

$$\exists e [\text{love}(e, P, F) \wedge \text{sweetly}(e)]$$

Approccio Neo-Davidsoniano

Il Neo-Davidsonismo (Parson) generalizza ulteriormente: **anche gli argomenti obbligatori** del verbo (non solo i modificatori opzionali come gli avverbi) diventano predicati binari sull'evento, separando completamente la struttura argomentale dal predicato principale:

“Paolo ama Francesca dolcemente” (slide Mazzei):

$$\exists e [love(e) \wedge agent(e, P) \wedge patient(e, F) \wedge sweetly(e)]$$

dove $agent(e, P)$ lega l'evento al suo agente (Paolo) e $patient(e, F)$ lega l'evento al suo paziente/tema (Francesca), esattamente con lo stesso meccanismo (predicato binario sull'evento) usato per l'avverbio $sweetly(e)$.

Vantaggi

- **Trattamento uniforme** di argomenti obbligatori e modificatori opzionali: gli avverbi diventano semplici predicati sull'evento come gli argomenti.
- **Inferenza**: da “Paolo ama Francesca dolcemente” si ottiene “Paolo ama Francesca” semplicemente per eliminazione del congiunto $sweetly(e)$ – l'esercizio di derivazione completa (albero sintattico + termini Lambda-FoL) per questa frase è proposto esplicitamente nelle slide di Mazzei.
- Integrazione naturale con risorse come **VerbNet** e **PropBank**, che usano ruoli tematici espliciti.
- Permette di aggregare informazioni provenienti da frasi diverse sullo stesso evento.

9. Le due ipotesi fondamentali della linguistica computazionale

Mazzei introduce il tema richiamando **Mark Steedman**, “On Becoming a Discipline” (ACL Anthology, J08-1008), che descrive la storia della linguistica computazionale come una **“grande guerra tra regole e statistica”** (*rules-driven vs. data-driven*), paragonata – con un'iperbole storica – alla contrapposizione tra teoria quantistica e relatività generale in fisica: due paradigmi che spiegano bene il loro dominio ma che storicamente hanno faticato a integrarsi.

I. Approccio rules-driven (ipotesi di regolarità):

Il linguaggio naturale è governato da *regolarità sistematiche* – fonologiche, morfologiche, sintattiche, semantiche – descrivibili tramite regole scritte a mano da un esperto linguista (regular expressions, grammatiche formali, automi a stati finiti).

II. Approccio data-driven (ipotesi statistica/empirica):

I dati linguistici reali (corpora) contengono informazione sufficiente per *apprendere, stimare e valutare* modelli del linguaggio tramite tecniche di Machine Learning (decision tree, regressione logistica, SVM, reti neurali), senza

scrivere regole esplicite.

Esempio guida (slide Mazzei): quando finisce una frase?

Per rendere concreto il dilemma, Mazzei pone la domanda: *come decidere se un punto “.” segna la fine di una frase (End of Sentence, EoS) o no?* (es. abbreviazioni come “Dott.”, numeri come “4.5”). Si tratta di un classificatore binario EoS/Not-EoS, che si può costruire in due modi:

- **By-hand** (rules-driven): regole scritte a mano, espressioni regolari, tokenizer a stati finiti + regole.
- **Machine Learning** (data-driven): un albero di decisione (es. C4.5) addestrato su un training corpus, con feature come “case della parola prima/dopo il punto”, “lunghezza della parola”, “probabilità che la parola preceda/seguiva un EoS”.

In entrambi i casi, secondo Mazzei, **il punto cruciale è la scelta delle feature linguistiche**: anche un albero di decisione appreso automaticamente non è altro che una serie di IF-THEN-ELSE, e la sua efficacia dipende dalla qualità delle feature scelte (che richiede comunque competenza linguistica).

Sintesi

- L’approccio **simbolico/rule-based** giustifica grammatiche, lessici, parser formali.
- L’approccio **statistico/neurale** giustifica n-gram, HMM, reti neurali, Transformer.

Nella linguistica computazionale moderna i due approcci si **integrano**: anche i modelli più statistici (PCFG, CRF, reti neurali) si basano su feature e strutture linguistiche definite da un esperto, e anche gli approcci rule-based più moderni usano corpora per validare le regole.

10. La Frame-Based Semantics nei Dialogue Systems (DS)

La **Frame-Based Semantics** è l’approccio dominante nella comprensione del linguaggio naturale (NLU) nei sistemi di dialogo orientati al task. Si basa sul concetto di **frame**: una struttura dati predefinita che rappresenta un’intenzione utente e i parametri necessari per soddisfarla.

Struttura di un frame

- **Intent (intenzione)**: l’azione che l’utente vuole compiere (es. `prenota_volo`).
- **Slot**: i campi parametrici da riempire (es. `ORIGINE`, `DESTINAZIONE`, `DATA`).
- **Tipo**: il tipo di valore atteso per ogni slot (città, data, orario, ...).
- **Valore**: il valore effettivo estratto dall’enunciato utente.

Esempio – frame `prenota_volo` (GUS, 1977):

Slot	Tipo	Domanda generata
ORIGINE	città	“Da quale città parti?”
DESTINAZIONE	città	“Dove vuoi andare?”
DATA_PARTENZA	data	“In che giorno vuoi partire?”
ORA_PARTENZA	orario	“A che ora preferisci partire?”
COMPAGNIA	compagnia	“Hai una compagnia preferita?”

Algoritmo in 3 passi (pipeline NLU)

1. **Domain Detection:** classificare l'enunciato nel dominio corretto (es. voli vs. hotel vs. meteo). Produce un insieme di frame candidati.
2. **Intent Detection:** selezionare il frame più probabile (es. prenota vs. cancella vs. cerca).
3. **Slot Filling:** estrarre i valori degli slot tramite NER e classificatori dedicati.

Gestione del dialogo (Dialogue Manager)

L'utente può riempire più slot in un unico enunciato:

“Voglio un volo da SF a Denver one-way, parto dopo le 17 di martedì”

→ ORIGINE=SF, DESTINAZIONE=Denver, TIPO=one-way, ORA_MIN=17:00, DATA=martedì

Quando tutti gli slot obbligatori sono riempiti il sistema esegue la query al database. Per gli slot mancanti, il sistema genera domande specifiche tramite **NLG**.

Condition-Action Rules sugli slot

Nei sistemi GUS, ogni slot può avere associate delle **regole condizione-azione** che si attivano automaticamente quando lo slot viene riempito. Esempi dalle slide:

1. Una volta che l'utente specifica la **DESTINATION** del volo → quella città viene automaticamente copiata come **StayLocation** di default per il frame di prenotazione hotel.
2. Una volta specificato il **DESTINATION DAY** per un viaggio breve → viene copiato automaticamente come **ARRIVAL DAY**.

Questo meccanismo permette al sistema di propagare informazione tra frame diversi senza richiedere all'utente di ripetersi.

Sistemi multi-frame e Frame Detection

Un sistema GUS reale gestisce **molteplici frame** contemporaneamente (prenotazione auto, hotel, informazioni su tratte, tariffe aeree, ecc.). Il sistema deve quindi risolvere il **frame detection task**: capire quale slot di quale frame l'utente sta riempiendo in un dato momento, e commutare il controllo del dialogo a quel frame. Questo si articola, secondo la pipeline NLU standard, in 3 passi – **Domain Classification** → **Intent Determination** → **Slot Filling** – esattamente come descritto sopra.

Limiti dei frame

I frame semplici non gestiscono bene task complessi con struttura ricorsiva (es. prenotare sia il volo *che* l'hotel). Soluzioni: gerarchie di frame, agenda di task.

11. Fare un esempio di ambiguità puramente semantica

L'ambiguità **puramente semantica** si manifesta quando una frase ha una struttura sintattica *unica* ma consente più interpretazioni semantiche.

Polisemia lessicale

“Ho visto **la banca**.”

→ Interpretazione 1: istituto finanziario.

→ Interpretazione 2: riva di un fiume.

La struttura sintattica (NP oggetto diretto) è identica in entrambi i casi; l'ambiguità è puramente nel significato del lessema “banca”.

Ambiguità di scope quantificazionale

“Ogni studente ha letto un libro.”

→ *Lettura 1* ($\forall > \exists$): $\forall x [stud(x) \Rightarrow \exists y [libro(y) \wedge legge(x, y)]]$

(ogni studente ha letto un libro qualsiasi, possibilmente diverso)

→ *Lettura 2* ($\exists > \forall$): $\exists y [libro(y) \wedge \forall x [stud(x) \Rightarrow legge(x, y)]]$

(c'è uno specifico libro che tutti gli studenti hanno letto)

Ambiguità del pronome (coreference)

“Mario ha detto a Luca che **lui** aveva ragione.”

→ “lui” = Mario *oppure* “lui” = Luca.

Esempio classico (Mazzei)

“I made her duck.”

- **Ambiguità morfologica:** *duck* = sostantivo (anatra) o verbo (abbassarsi).
- **Ambiguità sintattica:** *make* = causativo (→ “le ho fatto abbassare la testa”) o creativo (→ “le ho cucinato l'anatra”).
- **Ambiguità semantica:** *make* ha più significati.

12. Ambiguità sintattica: PP attachment e coordinazione

PP Attachment (Attacco del Sintagma Preposizionale)

Il **PP attachment** è uno dei fenomeni di ambiguità sintattica più studiati: un sintagma preposizionale (PP) può essere *attaccato* a costituenti diversi della frase, producendo interpretazioni diverse e quindi più alberi di derivazione per la stessa stringa.

Esempio canonico (slide Mazzei) – Groucho Marx, “Animal Crackers” (1930):

“One morning I shot an elephant in my pajamas. How he got into my pajamas I don’t know.”

- **Attacco al VP** (*shot*): ero io a indossare il pigiama mentre sparavo all’elefante.
[VP [V shot] [NP an elephant] [PP in my pajamas]]
- **Attacco all’NP** (*an elephant*): era l’elefante a indossare il pigiama (lettura assurda, ma sintatticamente valida – è la battuta comica di Groucho Marx).
[VP [V shot] [NP an elephant [PP in my pajamas]]]

La risoluzione è un problema *statistico e semantico*: richiede conoscenza delle restrizioni di selezione del verbo (selectional restrictions) e delle co-occorrenze lessicali, non risolvibile dalla sola sintassi.

Ambiguità della Coordinazione

La coordinazione con “e” (o *and*) può legarsi a costituenti di livello diverso, producendo letture diverse.

Esempi italiani (slide Mazzei):

- “...televisioni e radio nazionali...”
Lettura 1: [televisioni] e [radio nazionali] → solo le radio sono nazionali.
Lettura 2: [televisioni nazionali] e [radio nazionali] → entrambe nazionali.
- “Si può mangiare e bere qualcosa” vs. “Si può mangiare e pagare qualcosa”
Nel primo caso “qualcosa” è argomento condiviso di *mangiare* e *bere* (entrambi verbi che richiedono un oggetto del tipo cibo/bevanda); nel secondo, “pagare qualcosa” ha una struttura argomentale diversa da “mangiare”, e la condivisione dell’oggetto è meno naturale – la coordinazione forza una struttura sintattica che può risultare anomala semanticamente.

Crescita esponenziale delle analisi (ambiguità strutturale)

L’ambiguità della coordinazione, intrecciata con il PP attachment, produce un numero di alberi di derivazione che cresce in modo esponenziale con la lunghezza della frase. Con la grammatica giocattolo $S \rightarrow S S \mid a$ (che modella n PP/coordinazioni concatenate), il numero di analisi per n occorrenze di a segue i **numeri di Catalano**:

n (occorrenze di a)	1	2	3	4	5	6	7
# parse	1	1	2	5	14	42	132

Con appena 15 parole una grammatica ambigua può generare $\sim 10^6$ alberi di derivazione distinti: questo è il motivo per cui i parser reali non enumerano tutti gli alberi, ma usano modelli probabilistici (PCFG) per restituire solo l’analisi più probabile.

Nota terminologica: questi numeri di crescita sono i **numeri di Catalano** (citazione esatta delle slide: Church and Patil, 1982), che contano anche il numero di modi di incapsulare correttamente n coppie di parentesi (un’aperta precede sempre la sua chiusa corrispondente) – la stessa combinatoria della struttura ad albero.

13. Come funziona l'algoritmo CKY?

Il **Cocke-Kasami-Younger (CKY)** è un algoritmo di parsing a *chart* basato su programmazione dinamica per grammatiche in **Forma Normale di Chomsky (CNF)**.

CKY nell'anatomia del parser (Steedman)

Usando lo schema dell'**anatomia del parser** (vedi sezione dedicata), CKY si caratterizza come:

- **Grammar:** Context-Free (CNF).
- **Algorithm:** strategia di ricerca **bottom-up**; organizzazione della memoria a **programmazione dinamica** (chart), non backtracking.
- **Oracle:** probabilistico (MLE su treebank) nella versione PCKY, oppure assente nella versione di sola accettazione.

Perché bottom-up e non top-down? (Razionalisti vs. Empiricisti, slide Mazzei)

- **Top-down** (*goal-directed parsing*, “i Razionalisti”): si parte dalla grammatica, esplorando solo derivazioni che possono portare a S . Vantaggio: nessuna ricerca inutile rispetto all'obiettivo. Difetto: può costruire alberi sintatticamente legittimi ma **incompatibili con le parole effettive** della frase (li scopre solo alla fine).
- **Bottom-up** (*data-directed parsing*, “gli Empiricisti”): si parte dalle parole osservate, costruendo solo strutture compatibili con l'input. Vantaggio: mai sforzi sprecati su parole inesistenti. Difetto: può costruire alberi che poi risultano **non collegabili a S** (sintatticamente non validi globalmente).

CKY adotta la strategia bottom-up. Inoltre, un parser top-down naive incontra il problema della **ricorsione sinistra** (es. $NP \rightarrow NP PP$, espansione infinita senza consumare token) e dello spreco di lavoro sui **sotto-problemi condivisi** (lo stesso sotto-albero viene ri-derivato più volte per rami diversi) – entrambi i problemi motivano l'uso della programmazione dinamica.

Programmazione Dinamica (Bellman, 1957)

Principio di ottimalità (Bellman, 1957): “*An optimal policy has the property that whatever the initial state and initial decision are, the remaining decisions must constitute an optimal policy with regard to the state resulting from the first decision.*”

Si applica quando il problema ha: **sottostrutture ottimali** (la soluzione ottima si compone di soluzioni ottime dei sotto-problemi) e **sotto-problemi sovrapposti** (gli stessi sotto-problemi ricorrono più volte). La **memoizzazione** (memorizzare il risultato di ogni sotto-problema in tabella, evitando di ricalcolarlo) è la tecnica chiave – concettualmente simile al divide-et-impera, ma con riutilizzo esplicito dei risultati intermedi.

Pre-requisito: CNF

In CNF ogni regola ha una delle due forme: $A \rightarrow BC$ (due non-terminali) oppure $A \rightarrow w$ (un terminale). Qualsiasi CFG può essere convertita in CNF eliminando ε -produzioni, produzioni unitarie ($A \rightarrow B$) e binarizzando le produzioni con più di 2 simboli.

Struttura dati: tabella triangolare

Si costruisce una tabella $T[i][j]$ (con $0 \leq i < j \leq n$) che contiene l'insieme dei non-terminali capaci di derivare la sottostringa $w_{i+1} \cdots w_j$.

Inizializzazione (span di lunghezza 1, $j = i + 1$):

$$T[i][i+1] = \{A \mid A \rightarrow w_{i+1} \in R\}$$

Ricorsione (lunghezza $l = 2, \dots, n$; per ogni $i, j = i + l$):

$$T[i][j] = \bigcup_{k=i+1}^{j-1} \{A \mid A \rightarrow BC \in R, B \in T[i][k], C \in T[k][j]\}$$

Riconoscimento: la frase è grammaticale sse $S \in T[0][n]$.

Esempio: “Book that flight” (CKY)

Regole di esempio: $S \rightarrow NP VP$, $NP \rightarrow DET NOM$, $NOM \rightarrow N$, $VP \rightarrow V NP$, $V \rightarrow book$, $DET \rightarrow that$, $N \rightarrow flight$.

	[0, 1]	[1, 2]	[2, 3]
[0, 1]	V, NP	–	–
[1, 2]	–	DET	–
[2, 3]	–	–	N, NOM
[0, 2]	–	–	–
[1, 3]	–	NP ($\leftarrow$ DET + NOM)	
[0, 3]	VP ($\leftarrow$ V + NP), S ($\leftarrow$ NP + VP)		

$S \in T[0][3]$: la frase è riconosciuta.

14. Complessità dell'algoritmo CKY

Complessità temporale: $O(n^3 \cdot |G|)$

dove n è la lunghezza dell'input e $|G|$ è la dimensione della grammatica (numero di regole binarie).

Complessità spaziale: $O(n^2 \cdot |N|)$

dove $|N|$ è il numero di non-terminali.

Analisi dei loop

I tre loop annidati scorrono:

1. Lunghezza dello span l : n valori.
2. Posizione di inizio i : al massimo n valori.
3. Punto di split k : al massimo n valori.

Per ogni tripla (i, j, k) si verificano tutte le regole $A \rightarrow BC$: $O(|G|)$ operazioni.

Versione probabilistica (PCKY)

Attenzione: secondo le slide di Mazzei, la versione probabilistica del parsing a costituenti porta la complessità a $O(n^5)$ (non resta $O(n^3)$): questo è uno dei motivi per cui CKY/PCKY sono **troppo lenti per applicazioni real-time** (es. motori di ricerca). Le soluzioni pratiche sono: parsing parziale, pruning probabilistico (si scartano rami a probabilità troppo bassa durante la costruzione), oppure passare al parsing a dipendenze (più efficiente, vedi MALT $O(n)$).

Confronto con altri algoritmi (slide Mazzei)

Algoritmo	Caso peggiore	Note
CKY	$O(n^3 \cdot G)$	Richiede CNF. Difetto maggiore: caso peggiore
PCKY (probabilistico)	$O(n^5)$	Versione probabilistica del parsing a costituenti
Earley	$O(n^3)$ CFG generale	$O(n^2)$ se la grammatica non è ambigua; $O(n^3)$ altrimenti
Deterministic parsing (LL/LR)	$O(n)$	Un solo parse, ambiguità limitata – tipico
MALT (dipendenze)	$O(n)$	Greedy, non garantisce ottimo globale.
MST (dipendenze)	$O(n^2)$	Ottimo globale con Chu-Liu/Edmonds.

15. Dipendenze vs. Costituenti

Lucien Tesnière (1959), il fondatore della sintassi a dipendenze, la descrive così: *“The sentence is an organized whole, the constituent elements of which are words [...] The structural connections establish dependency relations between the words. Each connection in principle unites a superior term and an inferior term. The superior term receives the name **governor**. The inferior term receives the name **subordinate**.”* Esempio originale di Tesnière: nella frase “*Alfred parle*”, *parle* è il **governor** e *Alfred* il **subordinate**.

Aspetto	Costituenti	Dipendenze
Struttura	Albero con nodi interni (NP, VP...)	Albero con archi testa→dipendente
Nodi	Terminali + non-terminali	Solo terminali (le parole)
Head	Non esplicita (serve lessicalizzazione)	Esplicita per ogni arco
Ordine	Fortemente dipendente dall'ordine	Robusto a variazioni d'ordine
Relazioni	Struttura sintagmatica	Relazioni semantico-funzionali
Risorse	Penn Treebank (PTB), TIGER	Universal Dependencies (UD)
Parser	CKY, Earley, chart parser	MALT, MST, biaffine
Formalismo	CFG, PCFG, TAG	È il formalismo stesso

Vantaggi delle dipendenze (Tesnière 1959)

- Generalizzazione cross-linguistica: le relazioni testa-dipendente sono universali.
- Identificazione diretta delle relazioni predicato-argomento (superiore alla CFG per la semantica).
- Semplicità e trasparenza della struttura.
- Adatto a lingue a ordine libero (es. ceco, finlandese, latino).

Vantaggi dei costituenti

- Modellazione esplicita della struttura frasale.
- Integrazione naturale con grammatiche formali (CFG, CCG).
- Ricca tradizione e risorse (PTB con 1M parole annotate).

Convertibilità

Le due rappresentazioni sono parzialmente intercambiabili: le *head-percolation tables* permettono di estrarre dipendenze da un albero a costituenti, e viceversa. Le **Universal Dependencies** (UD) sono oggi lo standard multilinguistico per le dipendenze (100+ lingue).

16. La misura PARSEVAL

PARSEVAL è la metrica standard per valutare i parser a *costituenti* (Black et al. 1991).

Definizione

Dati l'albero prodotto dal parser (*proposed*) e l'albero di riferimento (*gold*), un **costituente** è identificato da una tripla (etichetta, inizio, fine). Si contano i costituenti proposti che coincidono con quelli gold.

$$\text{Precision} = \frac{|\text{proposti corretti}|}{|\text{proposti}|} \quad \text{Recall} = \frac{|\text{proposti corretti}|}{|\text{gold}|} \quad F_1 = \frac{2 \cdot P \cdot R}{P + R}$$

Labeled vs. Unlabeled

- **Labeled PARSEVAL (LP/LR)**: la tripla include l'etichetta del non-terminale (es. NP, VP).
- **Unlabeled PARSEVAL (UP/UR)**: conta solo la posizione dello span, ignorando l'etichetta.

Crossing Brackets

Una misura aggiuntiva usata da PARSEVAL è il **crossing brackets**: conta quante volte un costituente del system tree “incrocia” (si sovrappone parzialmente, senza essere né innestato né disgiunto) un costituente del gold tree. Esempio dalle slide:

System Tree: $((A\ B)\ C)$

Gold Tree: $(A\ (B\ C))$

Qui il costituente (AB) del system tree e il costituente (BC) del gold tree si sovrappongono parzialmente (condividono B ma non A né C nello stesso modo): è un crossing bracket. Un numero basso di crossing brackets indica che, anche quando il parser sbaglia l’attaccamento esatto, l’errore è “strutturalmente vicino” al gold standard.

Precision e Recall per le PCFG (in pratica)

Applicando le formule sopra a una PCFG:

- **Precision**: che percentuale dei subtree prodotti dal sistema sono anche nel gold tree? (“quanto di quello che ho prodotto è giusto”)
- **Recall**: che percentuale dei subtree del gold tree sono anche nel system tree? (“quanto di quello che avrei dovuto produrre ho effettivamente prodotto”)

Limitazioni

PARSEVAL non penalizza uniformemente tutti gli errori; è sensibile alle scelte di annotazione del treebank. Per la sintassi a dipendenze si usano:

- **UAS** (Unlabeled Attachment Score): % archi con testa corretta.
- **LAS** (Labeled Attachment Score): % archi con testa e relazione corretti.

17. Cosa è un Treebank?

Un **treebank** è un corpus di testo annotato con struttura sintattica (alberi), creato manualmente da linguisti esperti o semi-automaticamente (parser + revisione umana).

Caratteristiche

- Ogni frase è associata a un albero sintattico (a costituenti o a dipendenze).
- Le annotazioni includono spesso anche PoS tag e informazioni morfologiche.
- La creazione è costosa: nelle prime versioni, 1–2 ore per frase.

Esempi principali

Treebank	Lingua	Tipo	Dimensione
Penn Treebank (PTB)	Inglese	Costituenti	~1M token
Universal Dependencies (UD)	100+ lingue	Dipendenze	>200 treebank
Turin University Treebank (TUT)	Italiano	Dipendenze	–
Prague Dependency Treebank	Ceco	Dipendenze	~1.5M token

Universal Dependencies (UD): dettagli (slide Mazzei)

UD è oggi lo standard cross-linguistico, con **200 treebank in 100 lingue**. Caratteristiche distintive:

- Tokenizzazione *non* basata sugli spazi bianchi (a differenza, ad esempio, del TUT).
- Annotazione morfologica: Lemma + PoS + insieme di feature (genere, numero, ecc.).
- Si basa su **3 principi sintattici** (con ~40 relazioni di base): (1) le parole di contenuto sono collegate da relazioni di dipendenza; (2) le parole funzionali si attaccano alla parola di contenuto che specificano; (3) la punteggiatura si attacca alla testa della frase/clausola in cui appare.

Il **Turin University Treebank** (TUT, sviluppato a Torino, <http://www.di.unito.it/~tutreeb/>) è un esempio storico di treebank a dipendenze per l'italiano, citato esplicitamente nelle slide come precedente/alternativo a UD per la tokenizzazione basata sugli spazi.

Utilizzo

I treebank sono fondamentali per:

1. **Training** di parser supervisionati (MALT, biaffine, ...).
2. **Stima** dei parametri di PCFG via MLE.
3. **Valutazione** (gold standard per PARSEVAL, UAS/LAS).
4. **Ricerca linguistica** cross-linguistica (tramite UD).

18. HMM: modello generativo o discriminativo?

HMM è un modello *generativo*.

Spiegazione

Un modello generativo modella la **distribuzione congiunta** $P(X, Y)$ di osservazioni e etichette. Dall'HMM si può *generare* (campionare) una coppia (sequenza di tag, sequenza di parole) secondo il processo:

$$P(w_{1:n}, t_{1:n}) = P(t_1) \prod_{i=2}^n P(t_i | t_{i-1}) \prod_{i=1}^n P(w_i | t_i)$$

La classificazione avviene poi per inferenza bayesiana: $\hat{t} = \arg \max_t P(t | w) = \arg \max_t P(w, t)$.

Confronto con i modelli discriminativi

Modello	Tipo	Distribuzione modellata
HMM	Generativo	$P(X, Y)$
Naive Bayes	Generativo	$P(X, Y)$
MEMM	Discriminativo	$P(Y X)$
CRF	Discriminativo	$P(Y X)$ globale
SVM	Discriminativo	decision boundary

I modelli discriminativi modellano direttamente $P(Y | X)$, permettendo di usare feature arbitrarie; generalmente più accurati in classificazione. I generativi possono generare dati sintetici e gestire meglio i missing data.

19. Spiegare cosa si intende per Anatomia di un Parser

Con “**anatomia di un parser**” (Steedman 2000) si intende la scomposizione di qualsiasi sistema di parsing in tre componenti fondamentali:

1. Grammatica (Grammar / Model)

Specifica le strutture sintattiche ammissibili e i loro pesi. Può essere:

- Context-Free Grammar (CFG), PCFG, TAG, CCG, Grammatica a dipendenze.

2. Algoritmo (Algorithm)

La procedura di ricerca nello spazio delle strutture. Si articola in due dimensioni:

- *Strategia di ricerca*: top-down, bottom-up, left-to-right, beam search.
- *Organizzazione della memoria*: backtracking (DFS), programmazione dinamica (chart), greedy.

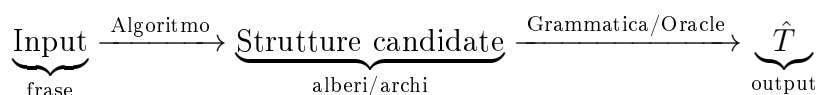
Esempi: CKY (bottom-up, DP), Earley (top-down, DP), MALT (greedy, nessuna DP).

3. Oracolo (Oracle)

Fornisce i “target” corretti durante il training. Può essere:

- *Rule-based*: determina univocamente la mossa corretta.
- *Probabilistico*: classifica le mosse tramite un modello ML addestrato su treebank.

Relazione tra i tre componenti



Esempi

Parser	Grammatica	Algoritmo	Oracolo
CKY	CFG/PCFG	Bottom-up, DP	MLE su treebank
MALT	Dipendenze	Greedy, transition	Classificatore ML
MSTParser	Dipendenze	CL/Edmonds	Arc-factored ML
Biaffine	Dipendenze	MST neurale	BiLSTM+biaffine

20. I 6 fenomeni notevoli del dialogo tra umani

Secondo Mazzei (lezione 10), i sei fenomeni fondamentali che caratterizzano il dialogo umano sono:

1. Turns (Turni)

Il dialogo è organizzato in *turni*: ogni contributo di un partecipante è un turno. Il **turn-taking** regola quando prendere, tenere o cedere la parola, ed è segnalato da cue prosodici, sintattici e gestuali. Sfide tecniche: gestione dell'interruzione (*barge-in*), rilevamento della fine del turno (*end-pointing*), pause interne al turno.

2. Speech Acts (Atti linguistici, Austin & Searle)

Ogni enunciato compie un'*azione* oltre a trasportare contenuto proposizionale. Quattro classi principali:

- **Constatives**: asseriscono qualcosa (answering, claiming, denying, stating).
- **Directives**: richiedono azioni all'interlocutore (asking, ordering, requesting, advising).
- **Commissives**: impegnano il parlante a un'azione futura (promising, planning, betting).
- **Acknowledgments**: esprimono atteggiamenti sociali (greeting, thanking, apologizing).

3. Grounding (Mutua comprensione)

I partecipanti costruiscono attivamente un *Common Ground*. Principio di **closure** (Clark 1996, dopo Norman 1988): un agente che compie un'azione ha bisogno di evidenza sufficiente di averla compiuta con successo – e **anche il parlare è un'azione**, quindi i parlanti devono “grounding-are” (confermare la comprensione) le affermazioni reciproche. Esempio dalle slide – perché i pulsanti dell'ascensore si illuminano quando premuti? Per dare un *grounding* visivo: confermare che la pressione è stata registrata.

Esempio di dialogo reale (trascrizione agente di viaggio – cliente):

- A: And you said returning on May 15th?
 C: Uh, yeah, at the end of the day.
 A: OK
 C: OK I'll take the 5ish flight on the night before on the 11th.
 A: On the 11th? OK.

Ogni “OK” è un atto di grounding: conferma che l’informazione precedente è stata registrata correttamente.

4. Dialogue Structure (Struttura del dialogo)

Il dialogo ha una struttura locale (*conversational analysis*, Sacks et al. 1974) e globale:

- **Adjacency pairs:** coppie di atti adiacenti (Question→Answer, Proposal→Acceptance/Reject, Compliment→Downplayer, es. “Nice jacket!” → “Oh, this old thing?”).
- **Sub-dialogues per correzione:** l’agente dice “There’s two non-stops”, il cliente interrompe (*overlap*, segnalato con #...#) per correggere un dettaglio, poi si torna al filo principale.
- **Sub-dialogues per chiarimento:** il sistema non riconosce una parola (“UNKNOWN WORD”) e chiede esplicitamente di ripetere/chiarire prima di proseguire.
- **Presequences:** sequenze preparatorie prima della richiesta principale (“Can you make train reservations?” → “Yes, I can.” → poi arriva la richiesta vera e propria).

5. Conversational Initiative (Iniziativa conversazionale)

Alcuni dialoghi sono controllati da una sola parte: l’esempio delle slide è un **giornalista che intervista uno chef** – il giornalista fa domande e lo chef risponde, quindi il giornalista ha l’iniziativa conversazionale (Walker & Whittaker, 1990). La maggior parte dei dialoghi umani ha invece **iniziativa mista** (*mixed initiative*): “io guido, poi tu guidi, poi io guido”. Gestire la mixed initiative è molto difficile per i sistemi NLP, che tendono a ricadere su stili più semplici e frustranti per l’utente: *user initiative* (l’utente chiede/comanda, il sistema risponde passivamente) oppure *system initiative* (il sistema pone domande per riempire un form, l’utente non può cambiare direzione).

6. Implicature (Grice 1975)

I parlanti comunicano più di quanto dicano letteralmente. Esempio dalle slide (un problema di inferenza “ancora più difficile”):

A: And, what day in May did you want to travel?

C: OK, uh, I need to be there for a meeting that’s from the 12th to the 15th.

Il cliente non risponde direttamente alla domanda (non dice un giorno preciso), ma l’agente deve **inferire** (implicatura conversazionale, massima di *relevance* di Grice) che il cliente vuole arrivare prima del 12 e partire dopo il 15. Le **massime conversazionali** di Grice sono: *quantità* (di’ abbastanza, non di’ troppo), *qualità* (di’ il vero), *relazione/relevance* (sii pertinente), *modo* (sii chiaro).

21. Cosa è il PoS Tagging?

Il **Part-of-Speech (PoS) Tagging** è il task di assegnare a ciascun token di una frase la sua **categoria grammaticale**, disambiguando i token ambigui in base al contesto. Esempio classico: “Plays well with others” ha ambiguità NNS/VBZ, UH/JJ/NN/RB, IN, NNS – l’output corretto è *Plays/VBZ well/RB with/IN others/NNS*.

A cosa serve (usi pratici, dalle slide)

- **Text-to-speech**: per decidere come pronunciare parole ambigue (es. “lead” si pronuncia diversamente come verbo o come nome del metallo).
- Scrivere espressioni regolari sull’output (es. (Det) Adj* N+) per identificare sintagmi.
- Input per velocizzare un parser completo.
- Traduzione automatica: riordinare aggettivi e nomi (es. spagnolo → inglese).
- Sentiment/affective analysis: distinguere aggettivi da altre categorie.
- Studio del cambiamento linguistico (nuove parole, slittamento di significato): bisogna controllare la PoS.

Classi principali e tagset

- **Classi aperte** (crescono con la lingua): sostantivi (NOUN), verbi (VERB), aggettivi (ADJ), avverbi (ADV).
- **Classi chiuse** (insieme finito): determinativi (DET), preposizioni (ADP), pronomi (PRON), congiunzioni, punteggiatura (PUNCT), ecc.

I due tagset di riferimento citati nelle slide sono il **Penn Treebank POS Tagset** (storicamente più usato per l’inglese, ~ 45 tag) e lo **Universal Dependencies Tagset** (Nivre et al. 2016, cross-linguistico).

Ambiguità e sfide

- Circa l’85% dei *tipi* lessicali è non ambiguo (“Janet” è sempre PROP, “hesitantly” è sempre ADV), ma quel 15% ambiguo tende a riguardare parole molto frequenti, per cui ~ 60% dei *token* in un testo reale è ambiguo. Esempio classico – “back”: *a back/ADJ seat, in the back/NOUN, senators back/VERB the bill, buy back/PART debt, twenty-one back/ADV then* – cinque categorie diverse per la stessa parola.
- **Fonti di informazione per disambiguare** (esempio: “Janet will back the bill”, dove “will” è AUX/NOUN/VERB e “back” è NOUN/VERB): probabilità a priori del tag per la parola (“will” è di solito AUX), identità delle parole vicine (“the” implica che la parola successiva probabilmente non è un verbo), morfologia/word-shape (prefissi, suffissi, capitalizzazione: “Janet” con maiuscola iniziale → PROP).

Approccio Rule-Based: ENGTWOL

ENGTWOL (*ENGLISH TWO Level analysis*) è il tagger rule-based di riferimento nelle slide, con un algoritmo in 2 stadi:

1. **Stadio 1**: si passano le parole attraverso un analizzatore morfologico che assegna *tutte* le PoS possibili. Esempio: “Pavlov had shown that salivation...” → Pavlov (N PROPER), had (HAVE V PAST / HAVE PCP2), shown (SHOW PCP2), that (ADV / PRON DEM / DET / CS – ambiguo!), salivation (N).

2. **Stadio 2:** si applicano regole in *forma negativa* (eliminano tag invece di assegnarli). Esempio – regola sull’avverbiale “that”: SE (la parola successiva è Agg/Avv/Quantificatore) E (la parola dopo ancora è un limite di frase) E NON (la parola precedente è un verbo come “consider”) ALLORA elimina tutti i tag non-ADV, ALTRIMENTI elimina ADV.

Tipicamente si usano più di 1000 regole scritte a mano (anche se possono essere apprese automaticamente).

Approcci statistici e neurali

- **HMM, MEMM/CRF:** vedi sezioni dedicate.
- **Neurali:** RNN/Transformer, LLM (BERT) fine-tuned.

Tutti richiedono un training set annotato a mano e raggiungono prestazioni simili (~ 97% su inglese). La **baseline** più semplice (tag più frequente per ogni parola + “unknown → NOUN”) raggiunge già ~ 92%, in parte perché molte parole non sono ambigue e perché si guadagnano punti gratis su articoli e punteggiatura.

Valutazione

Si confronta l’output con un **Gold Standard** (corpus annotato a mano), tipicamente diviso in: training data (80%), development data (10%), test data (10%).

22. Cosa è il NER Tagging?

Una **Named Entity**, nel suo uso più comune, è qualsiasi cosa a cui ci si possa riferire con un nome proprio. Il **Named Entity Recognition (NER)** è un task articolato in **2 sottotask**: (1) trovare gli *span* di testo che costituiscono nomi propri, e (2) etichettare il *tipo* dell’entità.

Classi più comuni (slide Mazzei/Jurafsky)

Etichetta	Esempio
PER (Person)	“Marie Curie”
LOC (Location)	“New York City”
ORG (Organization)	“Stanford University”
GPE (Geo-Political Entity)	“Boulder, Colorado”

Spesso sono sintagmi multi-parola. Il termine “entità nominata” viene talvolta esteso anche a cose che non sono entità in senso stretto: date, orari, prezzi.

Perché NER è più difficile del PoS Tagging

1. **Segmentazione:** nel PoS Tagging non c’è problema di segmentazione, perché ogni parola riceve esattamente un tag. Nel NER bisogna invece *trovare e segmentare* gli span (un’entità può occupare più token).
2. **Ambiguità di tipo:** la stessa stringa può appartenere a categorie diverse secondo il contesto (es. “Washington” = persona, città o stato?).

Schema di annotazione BIO

Per trasformare il problema strutturato (span + tipo) in un problema di sequence labelling con un'etichetta per token, si usa lo schema **BIO**: **B** (Begin, inizia uno span), **I** (Inside, interno a uno span), **O** (Outside, fuori da ogni span). Esempio (slide):

[PER Jane Villanueva] of [ORG United Airlines Holding], discussed the
[LOC Chicago] route.

diventa, con un tag per token: Jane/B-PER Villanueva/I-PER of/O United/B-ORG Airlines/I-ORG Holding/I-ORG ,/O discussed/O the/O Chicago/B-LOC route/O. Con n tipi di entità, il numero totale di tag è $2n + 1$: 1 tag O, n tag B, n tag I.

Approcci

- **Rule-based**: nella pratica commerciale, spesso liste (*gazetteers*) e regole pragmatiche con poco ML supervisionato – esempio citato: l'architettura **IBM System T**, dove l'utente specifica vincoli dichiarativi (regex, dizionari, vincoli semantici) compilati in un estrattore efficiente.
- **Statistici/Supervisionati**: HMM, MEMM/CRF – stesso impianto del PoS Tagging ma sullo schema BIO.
- **Neurali**: RNN/Transformer, LLM (BERT) fine-tuned.

Perché serve il NER

Sentiment analysis (qual è il sentiment verso una specifica azienda/persona?), Question Answering (rispondere a domande su un'entità), Information Extraction (estrarre fatti strutturati su entità dal testo).